Research Backup Architecture: Borg, Cloud Copies, and Time Machine

Ronald G. Thomas

2026-05-17



Figure 1: Three identical small wooden boxes stacked in a staggered row on a shelf, each a different size.

Three copies, two kinds of media, one off site: the rule is old, and the hard part is knowing that all three are still current.

Introduction

A research workstation accumulates material that cannot be regenerated: analysis code between commits, derived datasets that took days to compute, manuscripts in progress, and the credentials that unlock everything else. Version control protects some of this, but only what has been committed and pushed, and only the files that belong in a repository. A backup architecture covers the rest.

> **Note**
>
> Part 5 of the security series; start with Security Foundations for a Multi-Machine Research Fleet for the overview.

This post describes the backup architecture of a five-machine fleet: three Macs (julia, the primary workstation; athena, a laptop; joe, a secondary Mac), a Linux laptop (zeus), and a home server (mercury, a repurposed 2016 MacBook Pro running Linux Mint). It follows the familiar 3-2-1 rule with three independent mechanisms, each chosen for a different failure:

- **Borg** archives every machine nightly to the home server, encrypted, deduplicated, and versioned.
- **Cloud copies** mirror the project tree from the two main Macs to three cloud providers each night.
- **Time Machine** keeps a full, encrypted, browsable history of the laptop on a network share.

The design matters less than the operation. Every failure described near the end of this post was a backup that was scheduled, configured, and apparently working, and that had stopped doing its job. None of them produced an alert. All of them were found by reading logs and exit statuses by hand.

Motivations

- Git protects committed work, not the working tree, untracked data, or the dotfiles that hold credentials.
- A sync service is not a backup: it faithfully propagates deletions and corruption to every copy.
- A laptop can be lost, stolen, or wiped in an afternoon; the cost of recovery should be hours, not weeks.
- Credentials for one tier must not unlock another, so that a single compromised machine cannot destroy every copy.
- Backups that fail silently are worse than none, because they produce false confidence.

Objectives

1. Every machine is archived nightly to encrypted, versioned storage on the home server, through a key that can add data but not delete it.
2. The project tree on the main workstations has independent off-site copies with a recoverable history of deleted files.
3. The laptop has a full system history suitable for a bare-metal restore, encrypted at rest.
4. Old archives are pruned on a documented schedule without placing decryption keys on the storage host.
5. Each tier is verified by restoring from it, and each job's exit status is checked daily on every machine.

What is a Research Backup Architecture?

A backup architecture is a set of copies that fail independently. The 3-2-1 rule asks for three copies of the data, on two different kinds of storage, with one copy off site. In this fleet the mapping is:

Copy	Mechanism	Storage	Location	Protects against
1	The working files	Internal SSD	Each machine	(the original)
2	Borg	External disk on the home server	Home	Disk failure, deletion, ransomware on a client
3	rclone cloud copies	Google Drive, iCloud Drive, OneDrive	Off site	Loss of the house, the server, or both
(+)	Time Machine	External SSD on the home server	Home	Full-system loss of the laptop

Each mechanism answers a different question. Borg answers "what did this machine look like last Tuesday?" for every machine, with strong encryption and cheap history. The cloud copies answer "is there a copy of the projects somewhere other than this house?" Time Machine answers "how fast can the laptop be rebuilt exactly as it was?"

The tiers are deliberately unequal in scope. Borg covers the irreplaceable parts of every home directory, including secrets. The cloud copies cover only the project tree, because that is the material whose loss would be most costly and whose content is least sensitive. Time Machine covers one machine, completely, minus data that is regenerable or held elsewhere.

Prerequisites

- Borg 1.4 on every client and on the server (`brew install borgbackup` on macOS; the distribution package or the standalone binary on Linux).
- rclone 1.70 or later and `flock` (`brew install rclone flock`) on the machines that run cloud copies, with an rclone remote configured for each provider.
- A password manager that every client can read. This fleet uses `pass`; see Setting Up pass.
- A home server reachable at a stable address. This fleet uses Tailscale addresses for all scripted traffic; see A Home Server for a Five-Machine Research Fleet for the server itself.
- Full-disk encryption on every machine, so that the plaintext source of each backup is itself protected at rest.

Tier 1: Borg to the Home Server

Design

Each client writes to its own Borg repository on the home server: `/srv/backup/julia`, `/srv/backup/athena`, and so on. The home server in turn backs up its own home directory and the folders it serves to a repository on julia, so the server is never the only place its own data lives.

Three properties make this tier robust against a compromised client:

1. **Encryption with a per-repository passphrase.** Each repository uses Borg's `repokey` mode with its own passphrase. Every passphrase is stored in `pass`. The machine that physically holds a repository never holds its passphrase: the home server stores four clients' archives and can decrypt none of them, and julia holds the server's archive the same way.

2. **A restricted, append-only key per client.** Each client pushes with a dedicated SSH key that is good for nothing else. On the server, that key is pinned to a forced command:

   ```
   command="/usr/local/bin/borg serve --append-only \
     --restrict-to-repository /srv/backup/julia",restrict ssh-ed25519 \
     AAAA... borg-julia
   ```

   (Wrapped here for width; in `authorized_keys` it is one line.) The `restrict` option disables port forwarding, agent forwarding, and terminal allocation. `--restrict-to-repository` confines the key to one repository. `--append-only` means the key can add archives but cannot remove data already written. A thief or a piece of ransomware holding a client can write garbage into that client's repository; it cannot delete last month's archives.

3. **A separate SSH configuration for the backup.** This one is easy to get wrong. A typical `~/.ssh/config` already has an entry for the server with a general-purpose key. SSH offers keys from the config entry first, and `IdentityFile` is cumulative, so passing `-i` or `-o IdentityFile=` on the command line does not displace the general key. If the general key is accepted, the forced command never applies and the backup runs with full shell rights. The backup script therefore points Borg at its own configuration file:

   ```
   # ~/.borg/ssh_config (used only through BORG_RSH -F)
   Host mercury
     HostName <tailscale-address>
     User <user>
     IdentityFile ~/.ssh/id_ed25519_borg_julia
     IdentitiesOnly yes
     UserKnownHostsFile ~/.ssh/known_hosts
     StrictHostKeyChecking yes
     BatchMode yes
   ```

   `BatchMode yes` makes an unexpected prompt fail immediately rather than hang a scheduled run. The consequence of a second configuration file is that the server's address now lives in two places on each machine, a point that mattered later (see What broke, and how it was found).

The backup script

One script, identical on all five machines, is driven entirely by files in `~/.borg/`: the repository URL, the passphrase (mode 600), an include list, and an exclude list. The essential lines:

```
REPO=$(head -1 "$CONF/repo")
BORG_PASSPHRASE=$(cat "$CONF/passphrase")
export BORG_PASSPHRASE
export BORG_RSH="ssh -F $CONF/ssh_config -o ConnectTimeout=30"
# A differing URL makes borg ask an interactive question, which
# under a timer means a silent abort.
export BORG_RELOCATED_REPO_ACCESS_IS_OK=no

"$BORG" create \
  --stats --show-rc \
  --compression zstd,3 \
  --lock-wait 600 \
  --exclude-from "$CONF/exclude" \
  "$REPO::{hostname}-{now:%Y%m%d-%H%M%S}" \
  "${PATHS[@]}"
```

Three further details earn their place:

- **Missing include paths are logged, not fatal.** Each path in the include list is checked first. A path that does not exist, or that exists but cannot be listed, is skipped with a `WARNING` line. This keeps one stale entry from preventing the whole archive, but it creates its own risk: a partial archive still finishes with success. The first incident below is exactly this.
- **A watchdog.** A background timer kills the run after four hours. The failure that motivated it was Borg blocked on a cloud file-provider read for over half an hour at nearly zero CPU, holding the repository lock and wedging every later run.
- **No pruning from the client.** Under `--append-only`, a prune from the client only marks archives as deleted; space cannot be reclaimed. Retention runs on the server instead (below).

The script logs every run to `~/.borg/log/backup-<timestamp>.log`, ending with a single line `=== exit N`. Borg's exit codes are meaningful: 0 is success, 1 is success with warnings (a file changed while being read; the archive is valid), 2 is an error. The script adds 3 for local misconfiguration.

What is included

The include list names directories explicitly rather than archiving the whole home directory: the project tree (`~/prj`), documents (`~/docs`), the dotfiles repository, and the secrets that exist nowhere else: `.gnupg`, `.password-store`, `.aws`, `.Renviron`, and `.env` files. The Linux laptop archives its whole home directory with excludes. Caches, package libraries (`renv/library`, `node_modules`, virtual environments), and build artifacts are excluded everywhere.

Including `.gnupg` and `.password-store` may look redundant, since the same key and vault exist on several machines and the vault has its own git history. The reason is that replication is not backup:

an accidental deletion in the vault propagates to every machine at the next sync. A dated Borg archive is the copy that predates the mistake.

Scheduling

On the Macs a launchd user agent runs the script at a fixed time each night (02:00 on julia, staggered on the others so the server is not contended). On the Linux machines a systemd user timer does the same. Both hand the script a minimal `PATH`, which is why it locates the `borg` binary itself rather than trusting the login shell.

A Mac that is asleep at 02:00 runs the job at its next wake. If that wake is a brief maintenance wake on battery, the network may not be up yet, and the run fails on a connection timeout. A backup script should wait for, or retry, connectivity before declaring failure.

Retention

Without pruning, every repository grows without limit. The retention policy is:

```
borg prune --keep-daily 14 --keep-weekly 8 \
  --keep-monthly 12 --keep-yearly 3 "$REPO"
borg compact "$REPO"
```

Fourteen daily archives, eight weekly, twelve monthly, and three yearly: roughly two weeks of fine-grained history, a year of monthly checkpoints, and a few years of annual ones.

The policy runs **manually, once a quarter**, on the machine that holds each repository, not over the append-only path. This is a deliberate trade. Automating the prune would require each repository's passphrase in a file on the storage host, which gives up the property that the server cannot decrypt what it stores. A small script, `borg-prune`, makes the manual step cheap and safe: it prompts for the passphrase (read from `pass` on a machine that already holds it), lists the archives, shows a dry run of what the policy would delete, requires typing `YES`, and only then prunes and compacts. The pass over all five repositories is a standing item in the quarterly audit described in the series overview.



Figure 2: A single external hard drive resting on a desk beside a coiled cable and a small paper luggage tag.

Tier 2: Cloud Copies with rclone

Design

The two machines where most project work happens, julia and athena, each mirror `~/prj` nightly to Google Drive, iCloud Drive, and a personal OneDrive account. joe and zeus have no cloud job; Borg covers them, and three copies of two machines' projects are enough off-site redundancy.

Two decisions shape this tier:

- **One destination per machine.** Each machine syncs to its own folder, for example `backup/prj-julia` and `backup/prj-athena`. `rclone sync` makes the destination an exact mirror of the source, including deletions. When two machines mirrored their own slightly different copies of `~/prj` to the same folder, each run undid the other's, and neither copy was ever complete.
- **Deleted files are kept, by date.** `--backup-dir` moves anything the sync would delete or overwrite into a dated folder beside the mirror, so a file removed by mistake on Monday is still recoverable from the cloud on Friday.

The wrapper

All three targets go through one wrapper script, `backup-remote`, with a thin entry point per provider for launchd to call:

```
case "$target" in
  gdrive)
    dest="gdrive:backup/prj-$HOST"
    deleted="gdrive:backup/prj-${HOST}_deleted"
    lock=/tmp/rclone_gdrive.lock ;;
  # icloud and onedrive follow the same pattern
esac

# -E 75: a held lock means an earlier run is still going. Without it
# flock exits 1 silently, and a skipped night looks like a failure.
rc=0
/opt/homebrew/bin/flock -n -E 75 "$lock" \
  /opt/homebrew/bin/rclone sync -L "$PRJ_ROOT" "$dest" \
  --backup-dir "$deleted/$(date +%Y-%m-%d)" \
  --exclude "**/.cache/**" \
  --exclude "**/node_modules/**" \
  --exclude "**/.venv/**" \
  --exclude "**/renv/library/**" \
  "$@" || rc=$?

if [ "$rc" -eq 75 ]; then
  echo "$(date '+%Y/%m/%d %H:%M:%S') backup-remote: $target run" \
    "skipped: an earlier run still holds $lock" >&2
fi
exit "$rc"
```

The lock prevents two runs to the same destination from overlapping, which matters because the first upload to a provider can take many hours. The exit code 75 (`EX_TEMPFAIL`) makes a skipped run visible in `launchctl list` and leaves a dated line in the log.

Two hazards of `rclone sync`

Cache directories. The excludes are not an optimization. One night's Google Drive run went more than fifteen hours without finishing because it was uploading tens of thousands of small files from R package caches, which the provider deliberately throttles. The excludes now match the categories Borg already skips.

Symlinks and the deletion guard. `-L` follows symbolic links, which the project tree uses for shortcuts between repositories. If rclone hits any listing error during a run, it skips **all deletions** for that run as a safety measure, and logs `not deleting files as there were IO errors`. A single broken or self-referencing link anywhere in `~/prj` therefore disables deletion permanently and silently: files removed or renamed locally are never removed from the mirror, which grows without bound. A student repository containing a link that pointed to itself did exactly this on every machine that held it. The remedy is to scan for such links before uninstalling or moving anything:

```
# Links that loop or point nowhere, under the backed-up tree.
find -L ~/prj -type l 2>/dev/null
```

`find -L` reports links it cannot resolve; it is cheap enough to run in the quarterly audit.

Tier 3: Time Machine

Time Machine protects athena, the laptop, which is the machine most likely to be lost and the one where a full-system restore is most valuable. It backs up to a Samba share on the home server, on an external SSD. The server-side setup (the share, the disk, the firewall, and the binding problems that had to be solved) is described in A Home Server for a Five-Machine Research Fleet. This section covers policy.

Encryption is chosen once. Time Machine encrypts a network backup only if *Encrypt Backup* is selected at the moment the destination is added. The first attempt in this fleet was added without it and had written 258 GB before the omission was noticed; the only remedy was to remove the destination, delete the backup from the share, and start again. Generate the encryption password in the password manager first (`pass generate`), then paste it into the dialog, so it exists somewhere other than the backup's own Keychain entry.

Exclusions follow the same logic as the other tiers. Back up what cannot be regenerated or recovered from elsewhere; skip the rest. On athena that means excluding:

- `~/Downloads` and caches;
- Docker's data (images rebuild from `Dockerfile` and `renv.lock`);
- large vendor installations that their installer can restore;
- Messages attachments and previews, which Messages in iCloud already holds;
- the local copy of a server-held mail account;
- the Spotlight index, which rebuilds itself;
- the local copy of the cloud backup folder, which would otherwise be backed up twice.

`/Applications` and `/Library` stay in: with a minimal application set they are small, and they are what makes a restored system usable without reinstalling everything.

Exclusions are applied with `tmutil`, scripted so they can be reviewed and re-applied:

```
sudo tmutil addexclusion -p "$HOME/Downloads"
sudo tmutil addexclusion -p "$HOME/Library/Metadata/CoreSpotlight"
# -p records a fixed path. Check the result with:
defaults read /Library/Preferences/com.apple.TimeMachine SkipPaths
```

One trap: `tmutil isexcluded` reports a path as excluded if any parent is excluded. Once a parent such as `/Library` is included again, a script that relied on `isexcluded` to decide "already handled" will skip children that now need their own exclusion. Check the `SkipPaths` list or the item's own exclusion attribute instead.

Exclusions in a mail account folder are per account. Excluding the whole Mail folder would have dropped a local "On My Mac" mailbox that held recovered messages existing nowhere else. Exclude only the folder of an account whose mail is held on its server.

Restore Testing

A backup that has never been restored is a hypothesis. Each tier has a cheap test that belongs in the quarterly audit:

```
# Borg: list archives, verify repository consistency, and restore a
# known file into a scratch directory.
borg list "$REPO" | tail -3
borg check --repository-only "$REPO"
mkdir -p /tmp/restore-test && cd /tmp/restore-test
borg extract "$REPO::julia-20261001-020000" Users/me/docs/notes.md
```

- **Borg.** Pick a file that changed recently, restore it from the latest archive and from one a month old, and compare. Run `borg check` on each repository at least quarterly; a full `borg check --verify-data` reads every chunk and is worth an occasional overnight run.
- **Cloud copies.** Download a recently modified file from each provider and compare its checksum with the local file. Restore one file from a dated `_deleted` folder to confirm `--backup-dir` is working.
- **Time Machine.** Restore a file through the Time Machine browser. Before trusting the share for a bare-metal restore, restore a whole machine onto a spare disk once.

The archive name in the example is illustrative; use `borg list` to find a real one. Restores need the passphrase, so the test also confirms that the passphrase in `pass` still opens the repository.

Monitoring

There is no alerting in this fleet. The practice that replaces it is a short daily check of every job's last exit status and the freshness of its log, **on every machine, including the Linux ones**:

```
# macOS: the second column is the last exit status.
launchctl list | grep -E 'borg|rclone'

# Linux: the result of the last timer run.
systemctl --user show -p Result,ExecMainStatus borg-backup.service

# Either: the newest Borg log and its final lines.
f=$(ls -t ~/.borg/log/backup-*.log | head -1)
echo "$f"; grep -E '^=== ' "$f" | tail -2
```

A status of 0 everywhere and a log dated today is the whole check. Anything else gets read. The useful property of launchd's exit status column is that it already recorded every failure described below; nobody was looking at it. Automating this check, so that a non-zero status or a stale log produces a message, is the most valuable improvement left in the design.



Figure 3: The UCSD Geisel Library at dusk.

What broke, and how it was found

Every incident below was found by reading logs or exit statuses, most during the first quarterly audit. They are grouped by the lesson they teach.

A partial backup looks like a successful one

- **Coverage loss when a folder moved.** The Borg include lists on the Macs reached `~/prj`, `~/docs`, and a shared utilities folder indirectly, through the old sync folder they had once been symlinked into. When the sync client was uninstalled, that folder vanished, and every nightly run skipped all three paths with a one-line warning and finished with success. Archives shrank from roughly 68,000 to 88,000 files per machine to about 24,000. The include lists now name the directories directly. See Migrating Off Dropbox for the migration that caused it.
- **Secrets that were never included.** A review of the home directory found `.env`, `.Renviron`, and `.aws` in no backup at all. They were added, along with `.gnupg` and `.password-store`.

A job can fail every night without anyone noticing

- **A job that never ran.** A newly added OneDrive job on julia called a script through `~/bin`, a folder of links into the dotfiles repository, and the two new links had never been created. launchd recorded exit status 78 (program not found) every night.

- **A job silently unloaded.** After a restart, julia's Borg agent was no longer loaded in launchd. The job file was intact; the job simply was not scheduled. It was loaded again with `launchctl bootstrap`.
- **Invisible skips.** Before the `-E 75` change, a cloud run that found the lock held exited 1 and wrote nothing, indistinguishable from a real failure. julia's Google Drive copy skipped three nights this way while a long manual run held the lock.

The network is part of the backup

- **An address change broke Borg on three machines.** The home server rebooted and received new LAN addresses from the router. athena, joe, and zeus still pointed at the old address, and their Borg runs failed with connection timeouts for several nights. julia was unaffected because it already used the server's Tailscale address. The first fix updated `~/.ssh/config` and the runs still failed, because the Borg script reads its own `~/.borg/ssh_config`. The Linux laptop was found a day after the Macs, because the first status check covered only the Macs. All machines now use the Tailscale address in both files. Before trusting the new address, each machine confirmed that the server's host key matched the key previously recorded for the old one.

A fix needs checking on every machine it touches

- **The lock fix broke another machine.** The `-E 75` change calls `flock` at its Homebrew path. joe had never installed it, so every cloud run there now stopped immediately. Installing it revealed that joe had never had rclone or a remote configured either: its cloud jobs had been installed by a setup script and could never have worked. They were removed. The script would be sturdier if a missing lock tool produced a warning and an unlocked run.
- **Symlink loops.** Removing the self-referencing link on one machine fixed that machine. The same link existed in the same repository on two others.

Settings chosen once cannot be changed later

- **The unencrypted Time Machine backup.** Described above: encryption must be selected when the destination is added.

Things to Watch Out For

1. **Two SSH configurations, one address.** If the Borg script uses its own SSH config, any change to the server's address or key must be made in both files on every machine.
2. **A general key in `~/.ssh/config` defeats a restricted key** unless the backup uses `ssh -F` with `IdentitiesOnly yes`.
3. **`--append-only` means prune from the server.** A client-side prune appears to succeed and reclaims nothing.
4. **Borg's exit code 1 is not a failure,** and a missing include path is not an error. Read the `WARNING` lines, not just the exit code.

5. **`rclone sync -L` and broken links.** One listing error disables deletions for the run. Scan for loops before moving or uninstalling anything under the backed-up tree.
6. **Never point two machines at the same `rclone sync` destination.**
7. **Time Machine encryption is a setup-time choice.**
8. **`tmutil isexcluded` follows parents.** Check `SkipPaths` instead.
9. **Scheduled jobs can disappear.** Check that every agent or timer is still loaded, not only that its last run succeeded.
10. **A Mac's night job may run in a dark wake without a network.** Wait or retry before failing.

What Did We Learn?

Lessons Learned

- **Independent tiers fail independently, but they also fail silently.** Three mechanisms gave real redundancy; it was luck, not design, that no data was lost while one tier or another was broken.
- **Success status is not coverage.** Archive file counts, log warnings, and destination sizes say more about a backup than its exit code.
- **Separation of keys is cheap and effective.** Per-host repositories, restricted append-only keys, and passphrases kept off the storage host mean that no single machine can destroy or read the whole fleet's history.
- **Manual retention is an acceptable price** for keeping decryption keys off the server, provided it is on a schedule.
- **Stable addressing belongs to the backup design.** Scripted traffic should use addresses that do not change when a router reassigns a lease.

Limitations

- No automated alerting. Detection depends on someone running the daily check.
- The cloud tier covers only the project tree of two machines. The remaining home-directory content of every machine has two copies (the original and Borg), both in the same house, plus Time Machine for the laptop.
- The Borg repositories and the Time Machine share sit on the same server. A fire or theft at home leaves the cloud copies of the project tree as the only surviving tier.
- Restores are tested file by file; a full bare-metal restore is the test that matters most and the one least often performed.
- rclone's default Google Drive client identifier is shared and throttled; a dedicated API client is the durable configuration.

Opportunities for Improvement

1. A daily monitoring job that reports any non-zero exit status, any unloaded agent, or any log older than a day, on every machine.
2. Connectivity checks with retry at the start of each backup script.

3. An off-site copy of the Borg repositories themselves (for example, a second encrypted repository at a cloud provider), which would close the single-site limitation for the whole fleet.
4. A scripted quarterly restore drill that extracts a fixed sample of files from each tier and compares checksums.
5. The `flock` dependency made optional, so a missing lock tool degrades to a warning.

Wrapping Up

The architecture is simple to state: Borg nightly to a home server through keys that cannot delete, cloud copies of the projects with a history of deletions, and an encrypted Time Machine share for the laptop. The hard part was never the design. It was noticing, without any alert, that a tier had stopped working.

Main takeaways:

- Give every client its own repository, passphrase, and restricted append-only key, and keep passphrases off the storage host.
- Use `ssh -F` with a dedicated configuration so the restricted key is the one actually offered.
- Give each machine its own cloud destination and keep deleted files with `--backup-dir`.
- Choose Time Machine encryption when adding the destination; it cannot be added later.
- Check every job's exit status on every machine daily, and restore from every tier quarterly.

See Also

Security series:

1. Security Foundations for a Multi-Machine Research Fleet
2. Setting Up pass: a Unix Password Manager
3. Secrets Management for the Workflow Construct
4. Migrating Off Dropbox: Beyond Dotfiles
5. Research Backup Architecture: Borg, Cloud Copies, and Time Machine (this post)
6. A Home Server for a Five-Machine Research Fleet
7. Object Storage for Research Data: The Layer Git Cannot Hold
8. Running the Quarterly Security Audit for the First Time: What Actually Broke

External references:

- Borg documentation: `borg serve`, `--append-only`, and `prune`.
- rclone sync and the `--backup-dir` flag.
- Samba `vfs_fruit`: the module that serves Time Machine.
- `tmutil` manual page.

Reproducibility

The configuration excerpts are trimmed from the scripts in use on 2026-10-01; account names, addresses, and some excludes are replaced or omitted.

Component	Version
macOS	26.6
Linux Mint	22.3
Borg	1.4.5
rclone	1.75
flock (Homebrew)	0.4.0
Samba (server)	4.19 with `vfs_fruit`

The restore commands in the Restore Testing section use an illustrative archive name and path; substitute a real archive from `borg list`. Commands that require `sudo` are marked as such.

Let's Connect

Questions, suggestions, or corrections are welcome.

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas
- **Email:** Contact form

I would like to hear from you if:

- You find an error, or a better approach to anything in this series.
- You run a small research fleet and have found where this design breaks.
- You want to compare notes on backups, secrets, or reproducible research infrastructure.